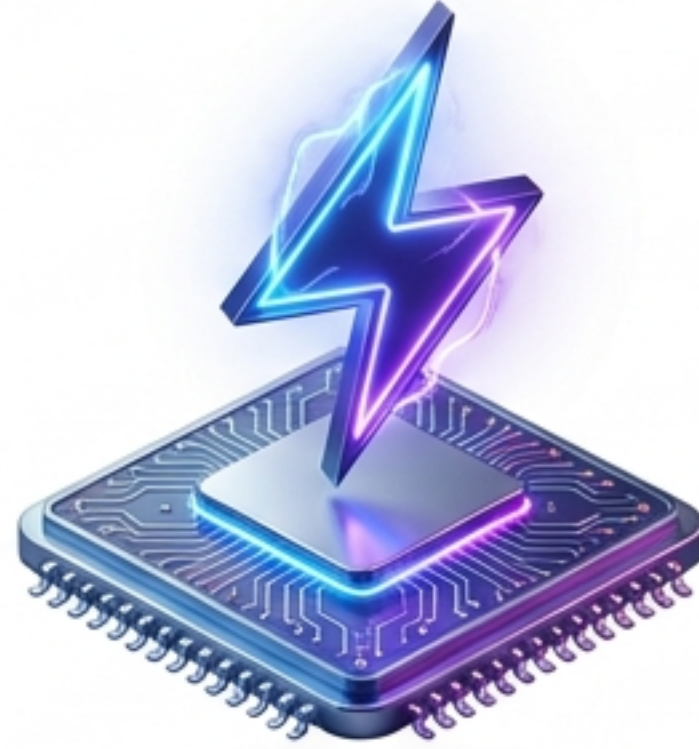




Tensors, PyTorch, JAX, Keras

Khám phá và So sánh các Framework Deep Learning Hiện đại

Tensors trong PyTorch

```
import torch

# Khởi tạo
x = torch.ones(size=(2, 1))
y = torch.zeros(size=(2, 1))
z = torch.tensor([1, 2, 3], dtype=torch.float32)

# Random (cú pháp hơi khác biệt)
r1 = torch.normal(mean=torch.zeros((3,1)),
                  std=torch.ones((3,1)))
r2 = torch.rand(3, 1) # Uniform
```

Khởi tạo cơ bản



Random (cú pháp hơi khác biệt)

Gán Tensor và nn.Parameter

Khác với TF, PyTorch Tensors có thể trực tiếp gán giá trị:

```
x = torch.zeros(size=(2, 1))  
x[0, 0] = 1.
```

Nhưng để theo dõi đạo hàm, ta sử dụng thuộc tính **requires_grad=True**:

```
v = torch.ones(size=(3, 1), requires_grad=True)  
# Hoac su dung Parameter khi viet Module  
import torch.nn as nn  
w = nn.Parameter(torch.rand(3, 3))
```



Tính toán Gradient bằng .backward()

Cú pháp đạo hàm của PyTorch đơn giản hơn
(không cần context manager GradientTape):



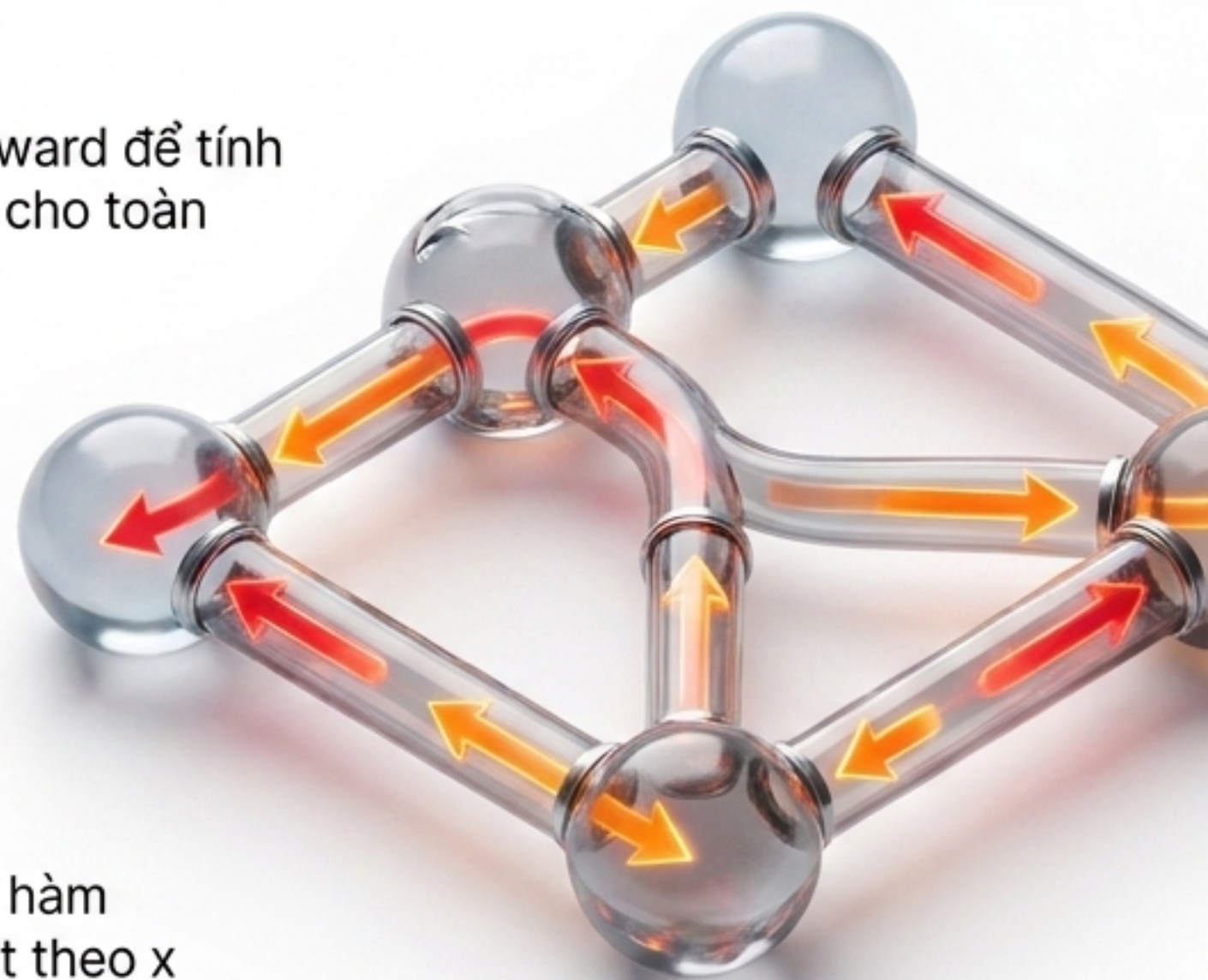
```
x = torch.tensor(3.0, requires_grad=True)
result = torch.square(x)

# Gọi backward để tính đạo hàm cho toàn bộ graph
result.backward ()

# Xem đạo hàm của result theo x
print(x.grad) # 6.0
```

Gọi backward để tính
đạo hàm cho toàn
bộ graph

Xem đạo hàm
của result theo x



Tổ chức code với nn.Module

```
class SimpleDense(nn.Module):  
    def __init__(self, in_channels, out_channels):  
        super().__init__()  
        self.W = nn.Parameter(torch.rand(in_channels, out_channels))  
        self.b = nn.Parameter(torch.zeros(out_channels))
```

Initialization
(State)

```
    def forward(self, x):  
        return torch.relu(torch.matmul(x, self.W) + self.b)
```

Execution
(Forward Pass)

```
model = SimpleDense(64, 32)
```

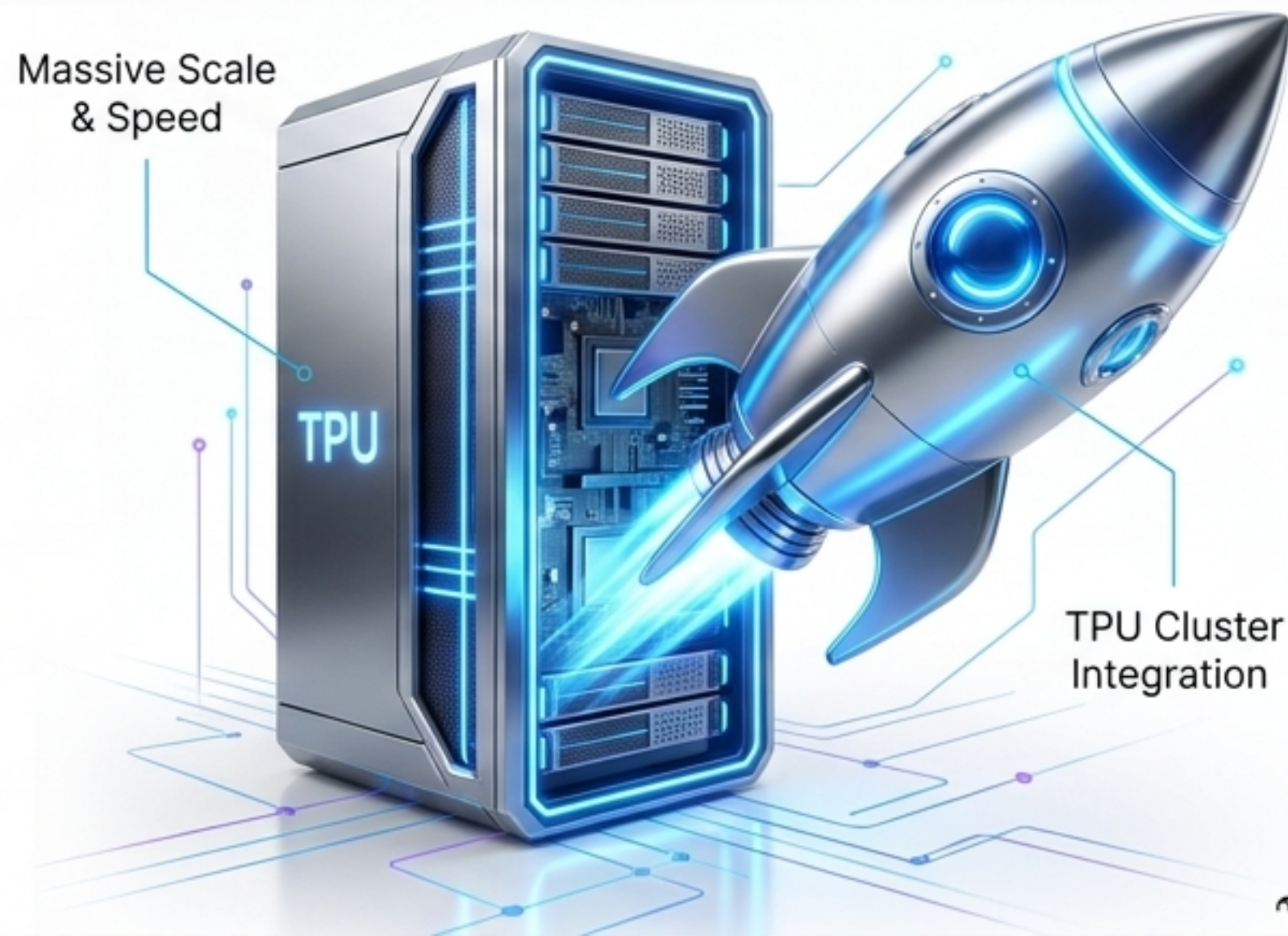


Giới thiệu JAX

$$[\text{Numpy}] + [\text{Autograd}] + [\text{XLA}] = [\text{JAX}]$$

Phổ biến ở Google DeepMind, Midjourney, Anthropic do khả năng chạy cực lớn trên cụm TPU.

Lập trình hàm không trạng thái (Stateless): Mọi hàm đều trong suốt (pure functions), không sử dụng biến toàn cục. Giúp JAX an toàn khi chạy song song.



jax.numpy và PRNG Keys

JAX tái tạo 99% API của Numpy thông qua **jax.numpy**:

```
from jax import numpy as jnp
x = jnp.ones(shape=(2, 1))
```

PRNG Keys (Số ngẫu nhiên không trạng thái):

```
import jax

# Phải tạo "chia khóa" (key) để random
seed_key = jax.random.key (1337)
r1 = jax.random.normal(seed_key , shape =(3,))

# Để lấy random tiếp theo , phải tạo key mới
new_key = jax.random.split(seed_key , num=1) [0]
```



Gán Tensor trong JAX

Giống TensorFlow, JAX tensor không thể gán trực tiếp. Phải dùng cú pháp `.at[].set()` để trả về một tensor mới:



```
x = jnp.array([1, 2, 3], dtype="float32")
```

```
# x[0] = 10 se bao loi
```

```
new_x = x.at[0].set(10)
```

Tính toán Gradient bằng jax.grad()

JAX sử dụng phương pháp "siêu lập trình" (hàm trả về hàm) để tính đạo hàm:

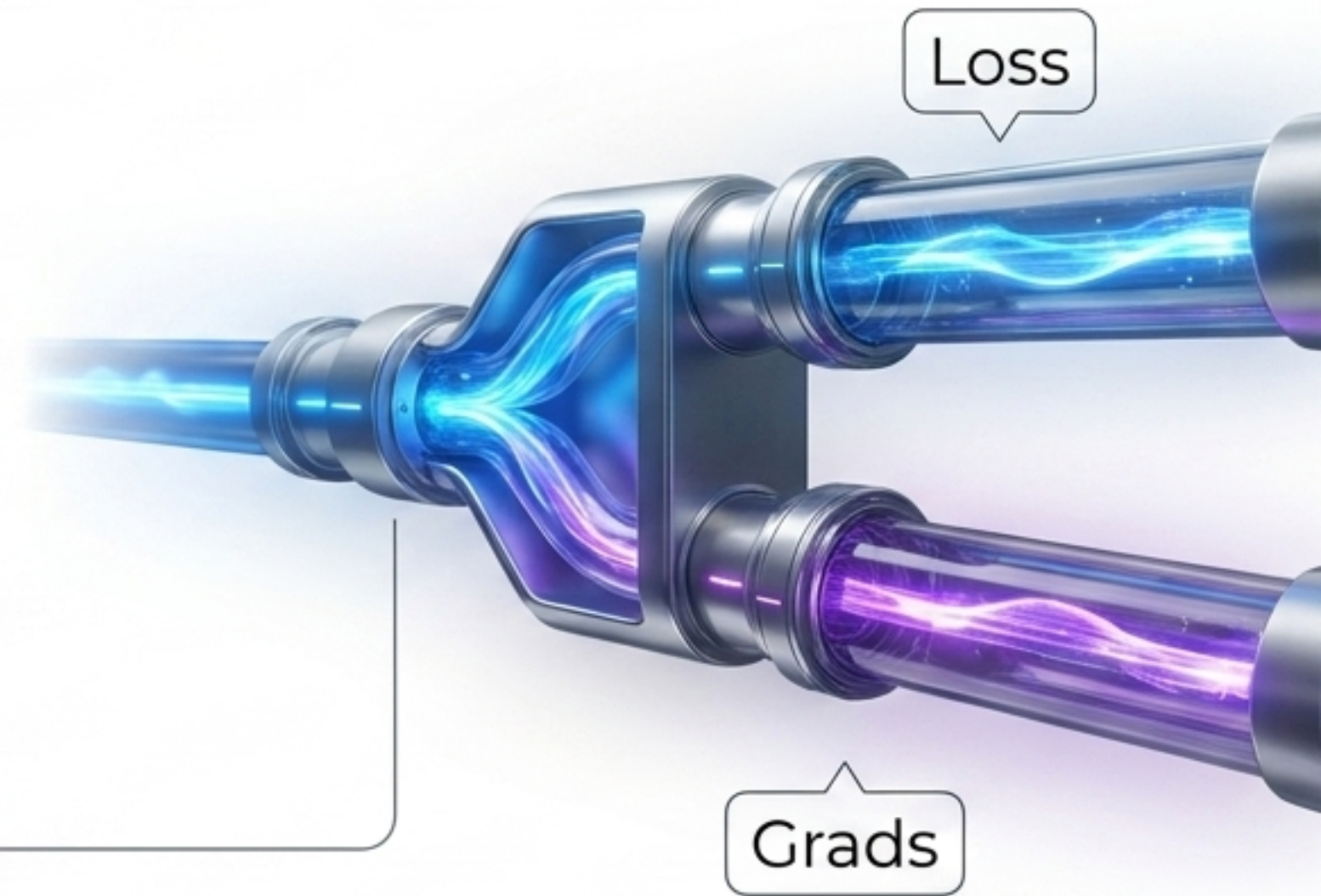
```
def compute_loss(input_var):  
    return jnp.square(input_var)  
  
# Truyền hàm loss qua jax.grad để lấy hàm  
# tính đạo hàm  
grad_fn = jax.grad(compute_loss)  
  
x = jnp.array(3.0)  
print(grad_fn(x)) # 6.0
```



Trả về cả Loss và Gradient

Thực tế, ta thường cần cả giá trị Loss và cấu trúc Gradient. JAX cung cấp **value_and_grad()**:

```
def compute_loss(state , x, y):  
    # state có thể là tuple (W, b)  
    pass  
  
# Trả về một hàm mới  
grad_fn = jax.value_and_grad(compute_loss)  
  
# Tính toán đồng thời  
loss , grads = grad_fn(state , x, y)
```



Biên dịch hàm cực nhanh với @jax.jit

```
@jax.jit
def dense(inputs , W, b):
    return jax.nn.relu(jnp.matmul(inputs , W) + b)
```



JAX biên dịch hàm này thành mã máy XLA.

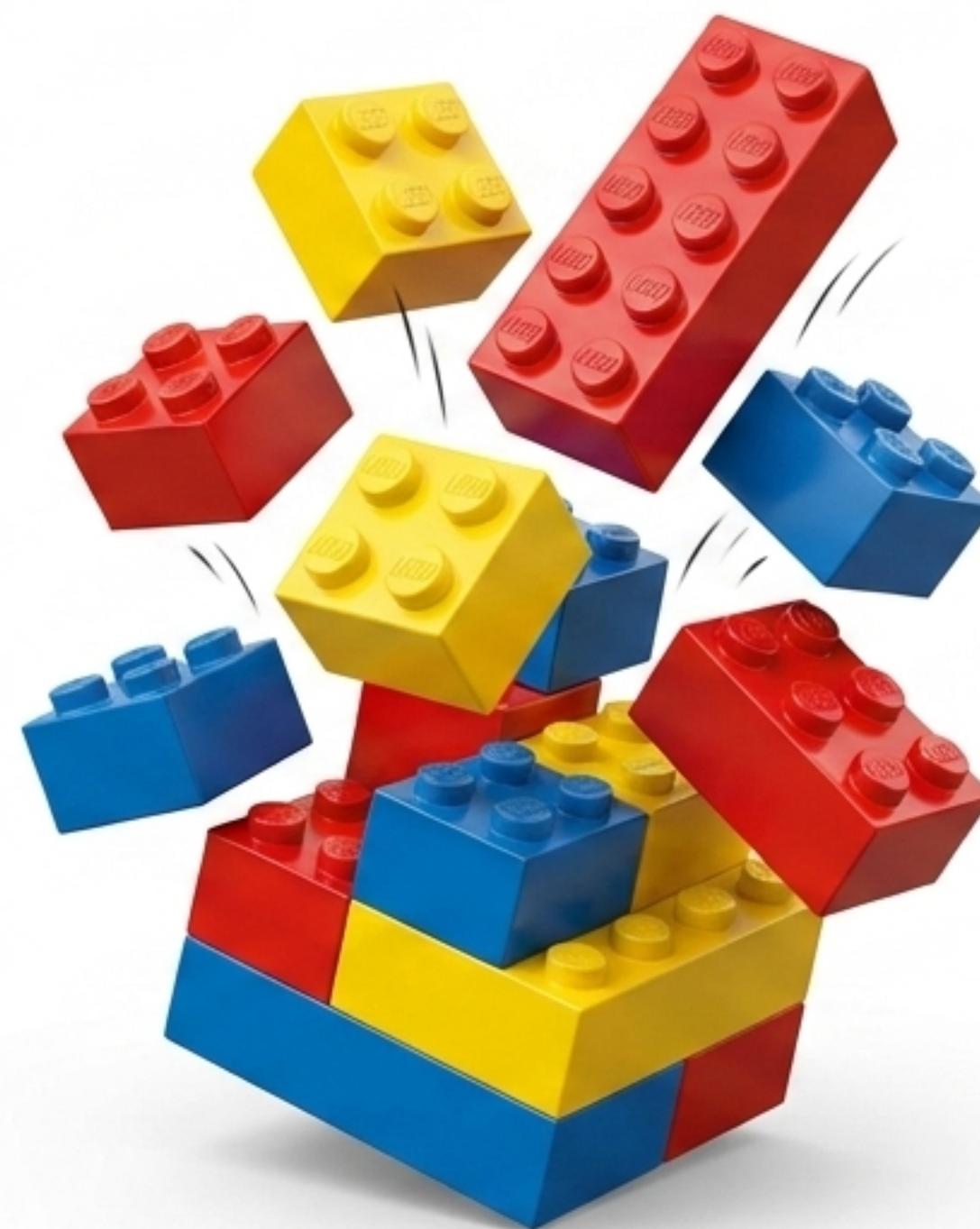
Ràng buộc: Hàm phải phi trạng thái (không sửa đổi biến ngoài, cập nhật tensor phải trả về).

Keras: High-Level API cho Deep Learning

Keras ra đời (2015) để giúp lập trình Deep Learning giống như chơi "lắp ghép LEGO".

Keras che đi toàn bộ sự phức tạp của Low-level API (viết Gradient, vòng lặp lặp).

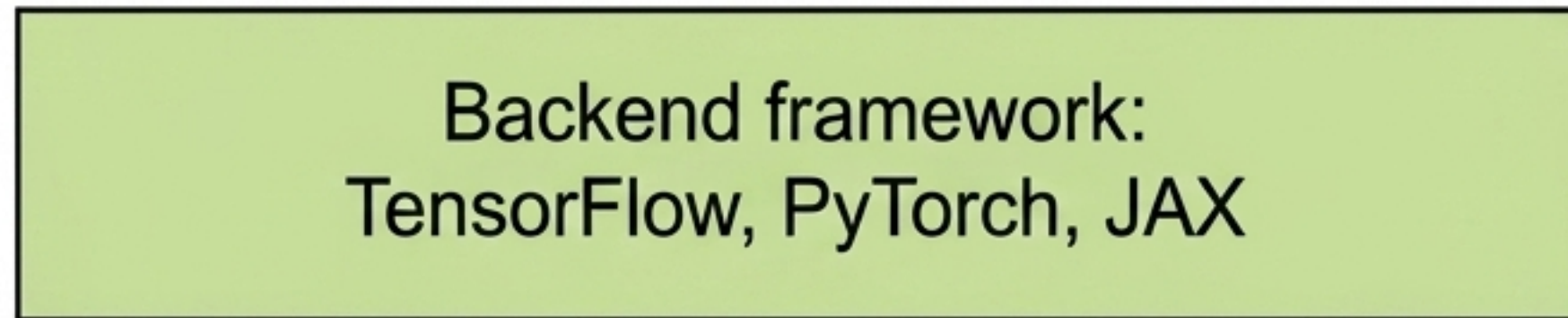
Được sử dụng bởi Google, Netflix, Uber, Waymo.



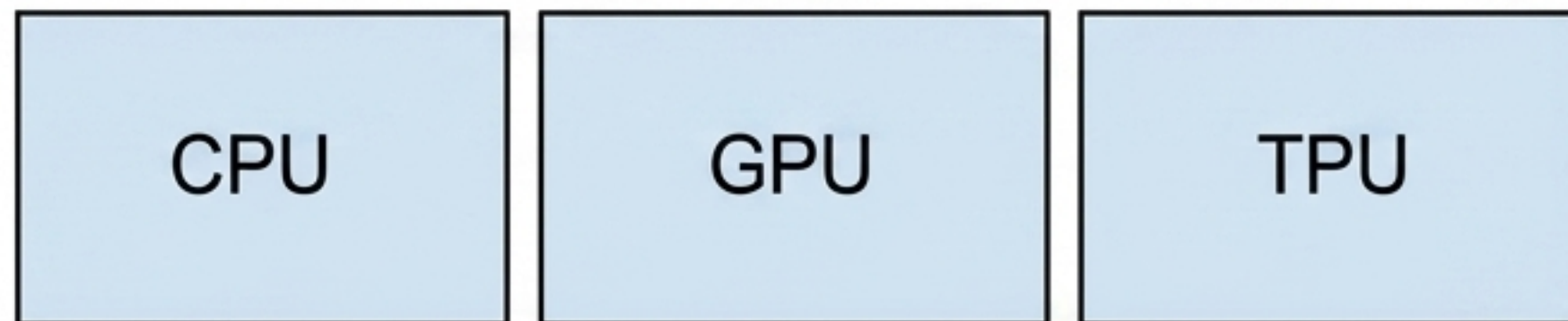
Kiến trúc đa Backend của Keras



Deep learning development:
layers, models, optimizers, losses,
metrics, training loops ...



Tensor manipulation infrastructure:
tensors, variables, automatic
differentiation, distribution ...



Hardware: execution

Hình 3.1: Cấu trúc đa Backend của Keras 3.0

Layer cơ sở: Thiết kế theo hướng Đối tượng

Trong Keras, Layer là khối xây dựng cơ bản, gồm trạng thái (weights) và tính toán (forward pass).

```
import keras

class SimpleDense(keras.Layer):
    def __init__(self, units):
        super().__init__()
        self.units = units

    def build(self, input_shape): # Gọi 1 lần duy nhất
        self.W = self.add_weight(shape=(input_shape[-1], self.units))
        self.b = self.add_weight(shape=(self.units,))

    def call(self, inputs): # Thực thi tính toán
        return keras.ops.matmul(inputs, self.W) + self.b
```

Lợi ích của hàm **build()**: Hàm **build()** cho phép Keras tự động suy diễn hình dạng (shape inference). Bạn không cần chỉ định đầu vào dài bao nhiêu:

```
# Bạn không cần quan tâm lớp trước truyền vào bao nhiêu unit
model = keras.Sequential([
    SimpleDense(32), # Tự động hiểu input_shape ở đây là 784
    SimpleDense(64),
    SimpleDense(10)
])
# Gọi model trên dữ liệu thực tế sẽ kích hoạt build() hàng loạt
```



Biên dịch Mô hình (model.compile)

Để cấu hình quá trình học tập, Keras có bước compile cực kỳ chuẩn hóa:

```
model.compile(  
    optimizer="adam", # Phép tối ưu gradient descent  
    loss="mse", # Hàm mất mát  
    metrics=["accuracy"] # Đo độ hiệu suất hiển thị  
)  
  
# Train trên dữ liệu  
model.fit(x_train, y_train, epochs=10)
```

Phép tối ưu
gradient descent

Hàm mất
mát

Độ đo hiệu
suất hiển thị

